

Towards a Characterisation of the Graph Structure of Process Interpretations of Regular Expressions

Clemens Grabmayer

<https://clegra.github.io>

Department of Computer Science



L'Aquila, Italy

TERMGRAPH 2026

Lisbon

July 19, 2026

Overview

- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ characterisation of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ characterization of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Overview

- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ characterisation of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ characterization of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Overview

- ▶ Regular expressions (unary/binary star, 1-free/[under-star-1-free](#) $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility question](#))
- ▶ Layered Loop existence and elimination ([LLEE](#)/[1-LLEE](#))
 - ▶ interpretation/[extraction](#) correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ [LLEE](#) is preserved by bisimulation collapse ([1-LLEE](#) is [not](#))
- ▶ TERMGRAPH 2024: [compact](#) process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is [not](#))
- ▶ [characterisation](#) of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ [characterization](#) of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Overview

- ▶ Regular expressions (unary/binary star, 1-free/[under-star-1-free](#) $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility question](#))
- ▶ Layered Loop existence and elimination ([LLEE](#)/[1-LLEE](#))
 - ▶ interpretation/[extraction](#) correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ [LLEE](#) is preserved by bisimulation collapse ([1-LLEE](#) is [not](#))
- ▶ TERMGRAPH 2024: [compact](#) process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is [not](#))
- ▶ [characterisation](#) of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ [characterization](#) of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Overview

- ▶ Regular expressions (unary/binary star, 1-free/**under-star-1-free** $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ **expressibility question**)
- ▶ Layered Loop existence and elimination (LLEE/**1-LLEE**)
 - ▶ interpretation/**extraction** correspondences with $(*/\perp)$ /**all** reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (**1-LLEE** is **not**)
- ▶ TERMGRAPH 2024: **compact** process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is **not**)
- ▶ **characterisation** of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ **characterization** of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Regular Expressions

Definition (*~ Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary Kleene star:

$e, e_1, e_2 ::= 0 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^*$ (for $a \in A$).

- ▶ symbol 0 instead of $\emptyset$

Regular Expressions

Definition (~ *Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary Kleene star:

$e, e_1, e_2 ::= 0 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**

Regular Expressions

Definition (*~ Kleene, 1951, ~ Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0** *
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Regular Expressions

Definition (*~ Kleene, 1951, ~ Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Regular Expressions (1-free)

Definition (*~ Kleene, 1951, ~ Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1 \otimes e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $\otimes$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

1-free regular expressions over alphabet A with binary Kleene star:

$$f, f_1, f_2 ::= 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1 \otimes f_2 \quad (\text{for } a \in A).$$

Regular Expressions (1-free)

Definition (*~ Kleene, 1951, ~ Copi-Elgot-Wright, 1958*)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\otimes} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $^{\otimes}$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

1-free regular expressions over alphabet A with unary / binary Kleene star:

$$f, f_1, f_2 ::= 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid (f_1^*) \cdot f_2 \quad (\text{for } a \in A),$$

$$f, f_1, f_2 ::= 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^{\otimes} f_2 \quad (\text{for } a \in A).$$

Regular Expressions (under-star-/1-free)

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star * : **1** is definable as **0^{*}**
- ▶ with binary star $^{\oplus}$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

The set $RExp^{(+)}(A)$ of **1-free regular expressions** over A is defined by:

$$f, f_1, f_2 ::=_{\text{assoc}(\cdot)} 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^* \cdot f_2 \quad (\text{for } a \in A),$$

Regular Expressions $((*/\perp)/1\text{-free})$

Definition ($\sim$ Kleene, 1951, $\sim$ Copi-Elgot-Wright, 1958)

Regular expressions over alphabet A with unary / binary Kleene star:

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e^* \quad (\text{for } a \in A).$$

$$e, e_1, e_2 ::= 0 \mid 1 \mid a \mid e_1 + e_2 \mid e_1 \cdot e_2 \mid e_1^{\oplus} e_2 \quad (\text{for } a \in A).$$

- ▶ symbol **0** instead of $\emptyset$, symbol **1** instead of $\{\epsilon\}$
- ▶ with unary star $*$: **1** is definable as **0^{*}**
- ▶ with binary star $\oplus$: **1** is **not** definable (in its absence)

Definition (for process interpretation)

The set $RExp^{(+)}(A)$ of **1-free regular expressions** over A is defined by:

$$f, f_1, f_2 ::=_{\text{assoc}(\cdot)} 0 \mid a \mid f_1 + f_2 \mid f_1 \cdot f_2 \mid f_1^* \cdot f_2 \quad (\text{for } a \in A),$$

the set $RExp^{(*/\perp)}(A)$ of **under-star-1-free $((*/\perp)$ reg. expr.** over A by:

$$uf, uf_1, uf_2 ::= 0 \mid 1 \mid a \mid uf_1 + uf_2 \mid uf_1 \cdot uf_2 \mid f^* \quad (\text{for } a \in A).$$

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P}$ deadlock δ , no termination

$0^* \xrightarrow{P}$ empty-step process ϵ , then terminate

$a \xrightarrow{P}$ atomic action a , then terminate

$e_1 + e_2 \xrightarrow{P}$ (*choice*) execute $P(e_1)$ or $P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P}$ (*sequentialization*) execute $P(e_1)$, then $P(e_2)$

$e^* \xrightarrow{P}$ (*iteration*) repeat (terminate or execute $P(e)$)

$\llbracket e \rrbracket_P := [P(e)]_{\Leftrightarrow}$ (*bisimilarity* equivalence class of process $P(e)$)

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \quad (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \quad (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \qquad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \qquad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \qquad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \qquad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \qquad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \quad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \quad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \quad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \quad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \quad \frac{e_1 \Downarrow \quad e_2 \xrightarrow{a} e'_2}{e_1 \cdot e_2 \xrightarrow{a} e'_2} \quad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \quad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \quad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \quad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \quad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \quad \frac{e_1 \Downarrow \quad e_2 \xrightarrow{a} e'_2}{e_1 \cdot e_2 \xrightarrow{a} e'_2} \quad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

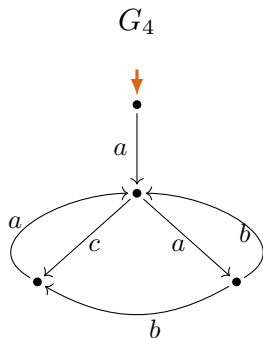
Definition

The **process interpretation** $P(e)$ of a regular expression e :

$P(e) :=$ **labeled transition graph** generated by e by derivations in $\mathcal{T}$,

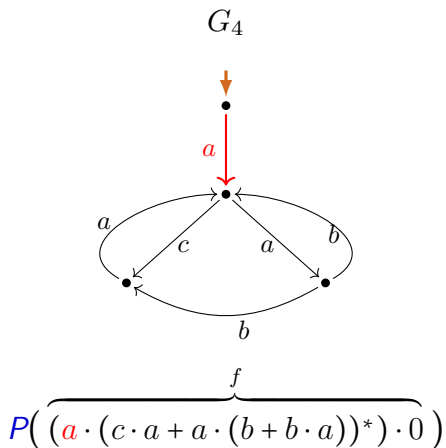
The **process semantics** $\llbracket e \rrbracket_P$ of e is: $\llbracket e \rrbracket_P := [P(e)]_{\leftrightarrow}$.

P -expressibility and $[[\cdot]]_P$ -expressibility

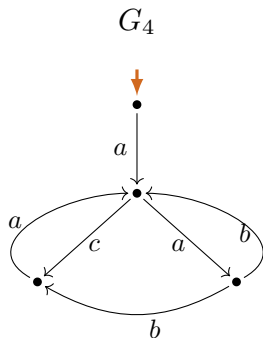


$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^*)}^f \cdot 0 \right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility

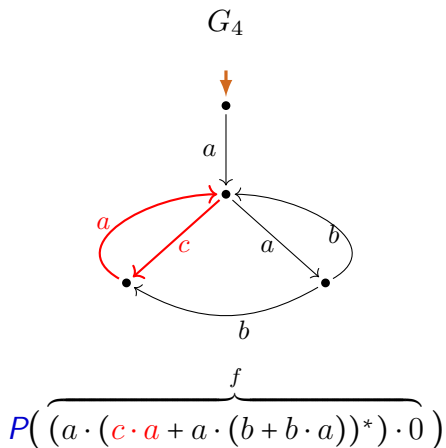


P -expressibility and $[[\cdot]]_P$ -expressibility

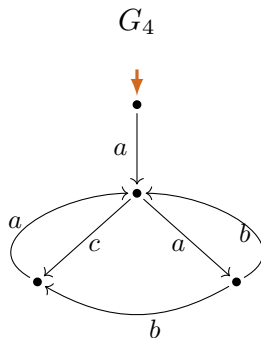


$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^*)}^f \cdot 0\right)$$

P -expressibility and $[[\cdot]]_P$ -expressibility

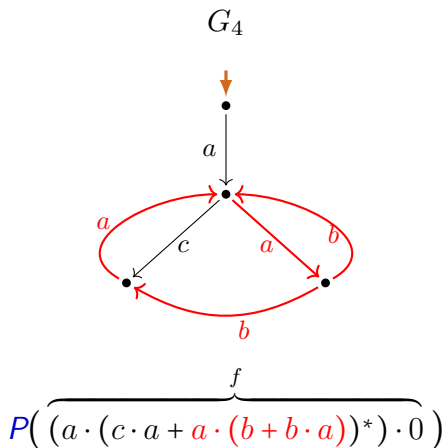


P -expressibility and $[[\cdot]]_P$ -expressibility

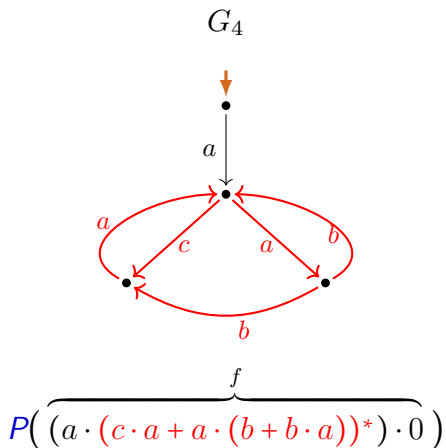


$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^*)}^f \cdot 0\right)$$

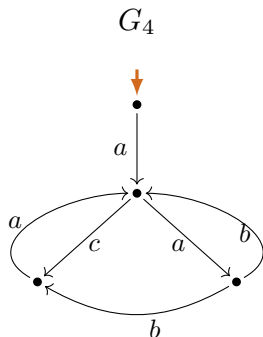
P -expressibility and $[[\cdot]]_P$ -expressibility



P -expressibility and $[[\cdot]]_P$ -expressibility



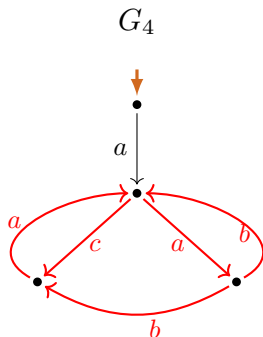
P -expressibility and $[[\cdot]]_P$ -expressibility



$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a)))^*}^f \cdot 0 \right)$$

$$P\left(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^{\otimes} 0 \right)$$

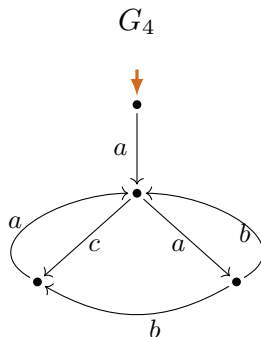
P -expressibility and $[[\cdot]]_P$ -expressibility



$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^*) \cdot 0}^f\right)$$

$$P\left(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^{\otimes} 0\right)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility

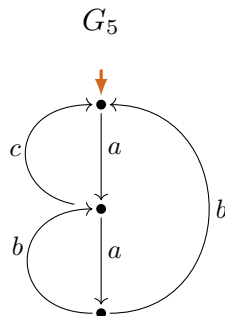
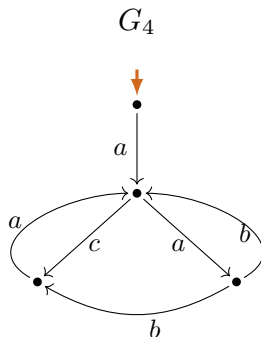


$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a)))^* \cdot 0}^f \right)$$

$$P\left(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^{\otimes} 0 \right)$$

$$G_4 \in \llbracket f \rrbracket_P$$

P -expressibility and $[[\cdot]]_P$ -expressibility

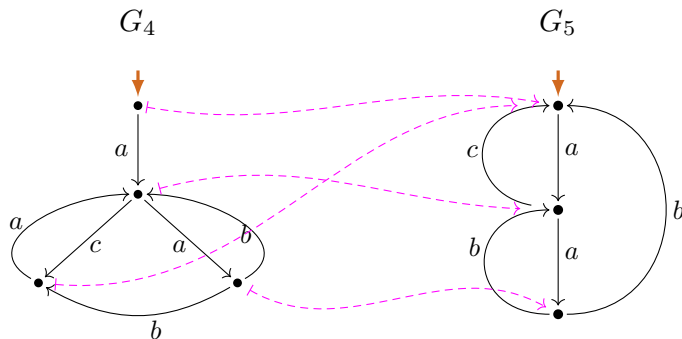


$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a)))^*}^f \cdot 0 \right)$$

$$P\left(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^{\otimes} 0 \right)$$

$$G_4 \in [[f]]_P$$

P -expressibility and $[[\cdot]]_P$ -expressibility

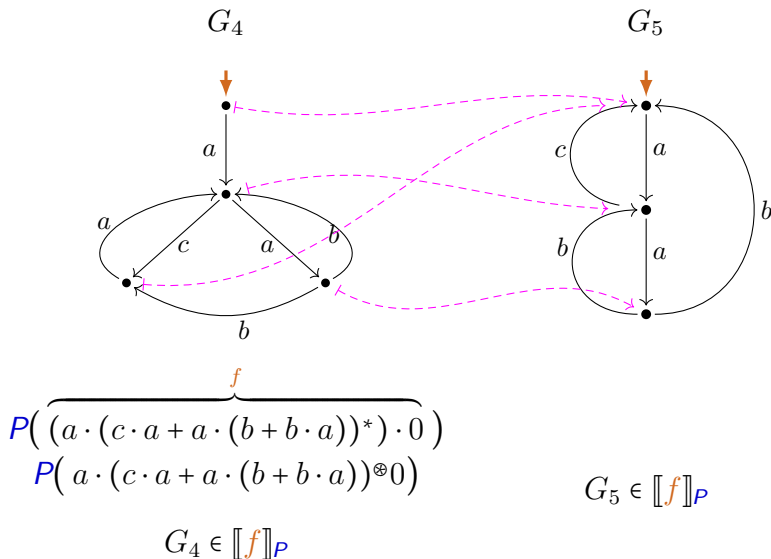


$$P\left(\overbrace{(a \cdot (c \cdot a + a \cdot (b + b \cdot a)))^* \cdot 0}^f\right)$$

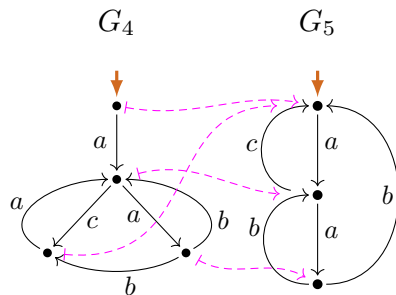
$$P\left(a \cdot (c \cdot a + a \cdot (b + b \cdot a))^{\otimes} 0\right)$$

$$G_4 \in [[f]]_P$$

P -expressibility and $[[\cdot]]_P$ -expressibility

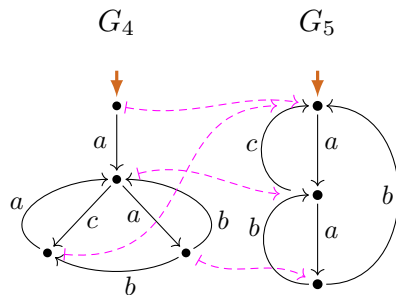


P -expressibility and $[[\cdot]]_P$ -expressibility/ (examples)



$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

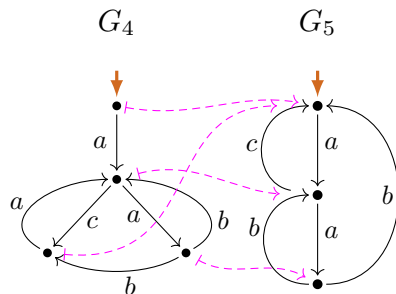
P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)



P -expressible

$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)



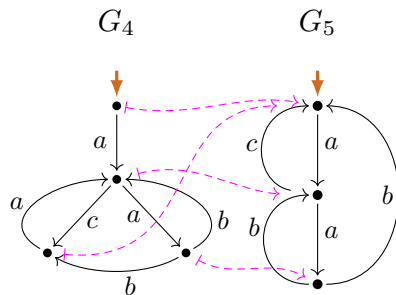
P -expressible

$\llbracket \cdot \rrbracket_P$ -expressible

$\llbracket \cdot \rrbracket_P$ -expressible

$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)



P -expressible

$\llbracket \cdot \rrbracket_P$ -expressible

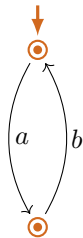
P -expressible?

$\llbracket \cdot \rrbracket_P$ -expressible

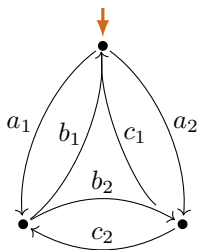
$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

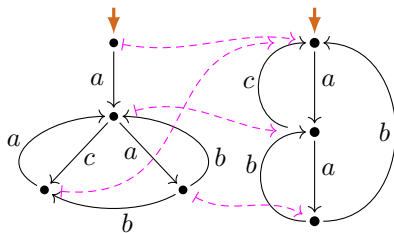
G_1



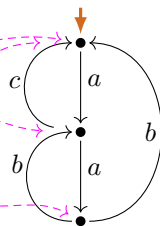
G_2



G_4



G_5



P -expressible

$\llbracket \cdot \rrbracket_P$ -expressible

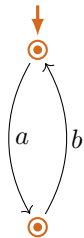
P -expressible?

$\llbracket \cdot \rrbracket_P$ -expressible

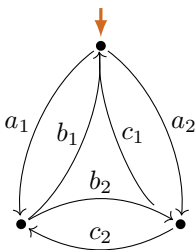
$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

G_1

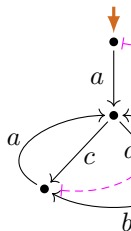


G_2



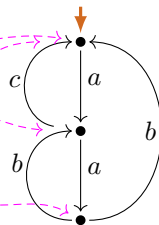
not P -expressible

G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

G_5

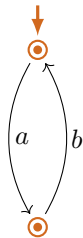


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

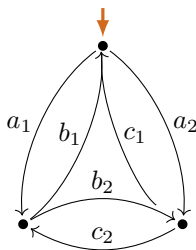
$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^*) \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

G_1

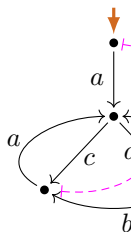


G_2



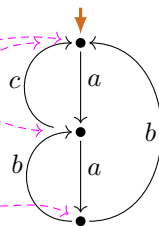
not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

G_5

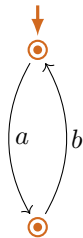


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

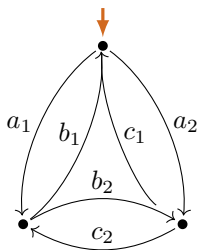
P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

G_1

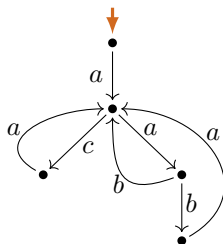


not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

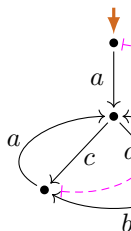
G_2



G_3

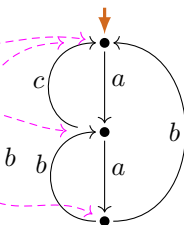


G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

G_5

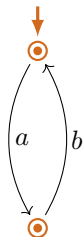


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

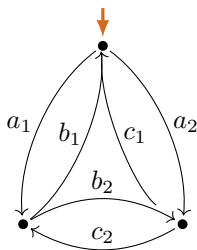
$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

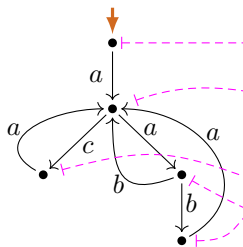
G_1



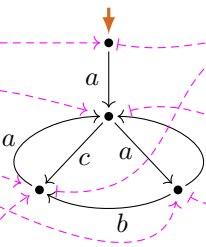
G_2



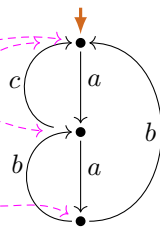
G_3



G_4



G_5



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

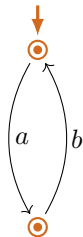
P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

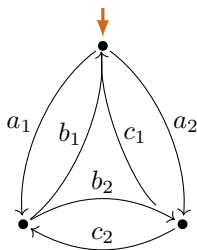
$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

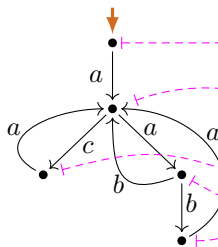
G_1



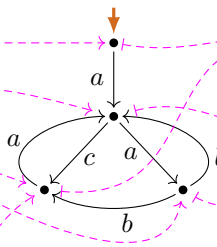
G_2



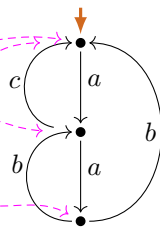
G_3



G_4



G_5



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

$\llbracket \cdot \rrbracket_P$ -expressible

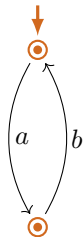
P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

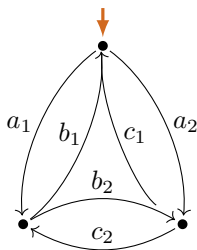
$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0)$$

P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

G_1

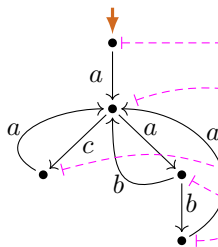


G_2



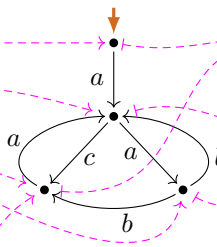
not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

G_3



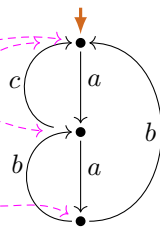
P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

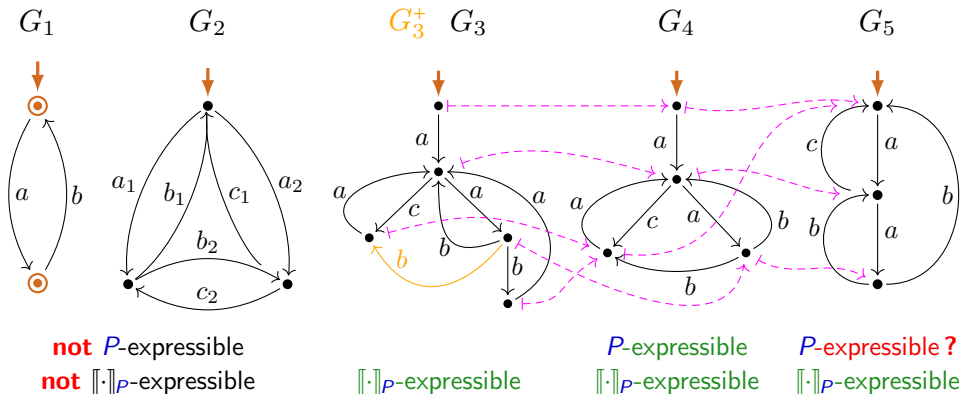
G_5



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

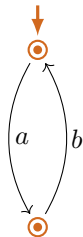
P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)



$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

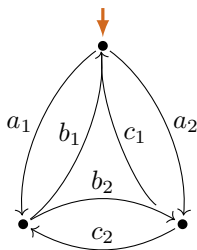
P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

G_1

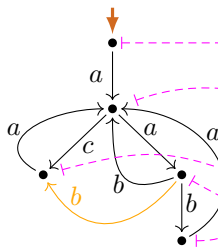


not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

G_2

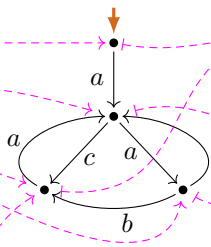


G_3^+ G_3



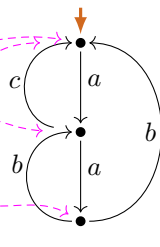
P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

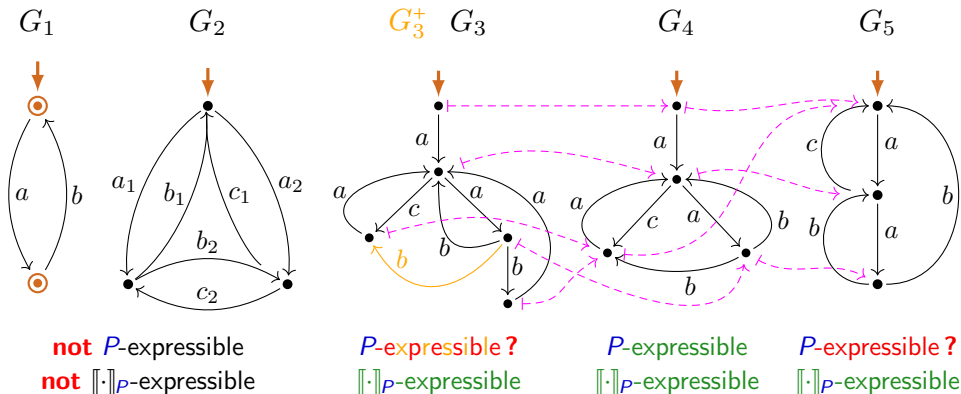
G_5



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

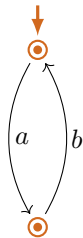
P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)



$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

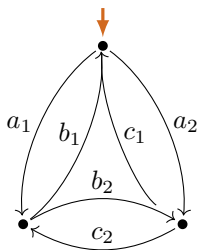
P -expressibility and $\llbracket \cdot \rrbracket_P$ -expressibility/ (examples)

G_1

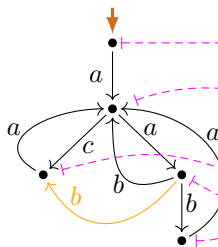


not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

G_2

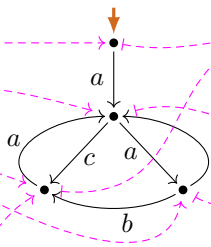


G_3^+ G_3



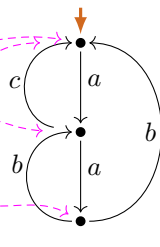
P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

G_5



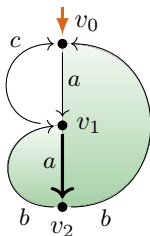
P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

$$P((a \cdot (c \cdot a + a \cdot (b + b \cdot a))^* \cdot 0))$$

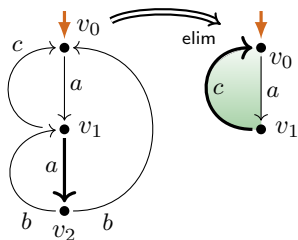
LLEE and 1-LLEE

- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ characterisation of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ characterization of image of $P^\bullet$
- ▶ Relationship with co-reducibility

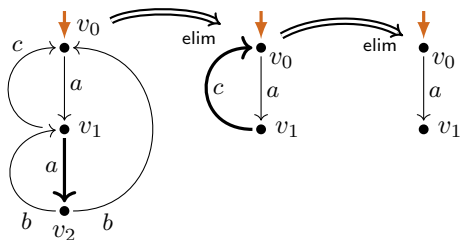
Loop existence and elimination



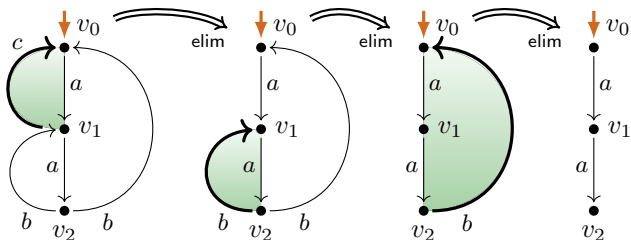
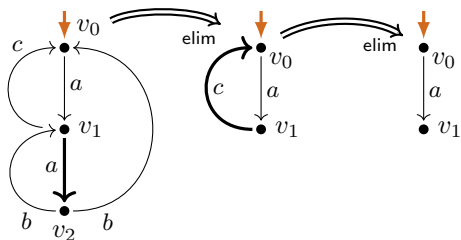
Loop existence and elimination



Loop existence and elimination



Loop existence and elimination



L

Definition

A graph G satisfies LEE (*loop existence and elimination*) if:

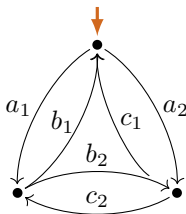
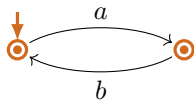
$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\rightarrow_{\text{elim}} \wedge G_0 \text{ permits no infinite path} \right).$$

L EE

Definition

A graph G satisfies **L EE** (*loop existence and elimination*) if:

$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\rightarrow_{\text{elim}} \right. \\ \left. \wedge G_0 \text{ permits no infinite path} \right).$$

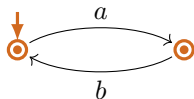


LLEE

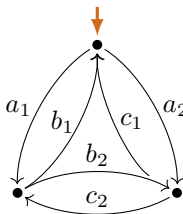
Definition

A graph G satisfies **LLEE** (*loop existence and elimination*) if:

$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\rightarrow_{\text{elim}} \wedge G_0 \text{ permits no infinite path} \right).$$

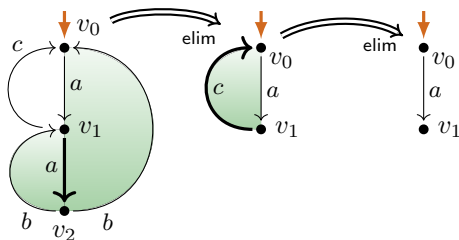


not LEE

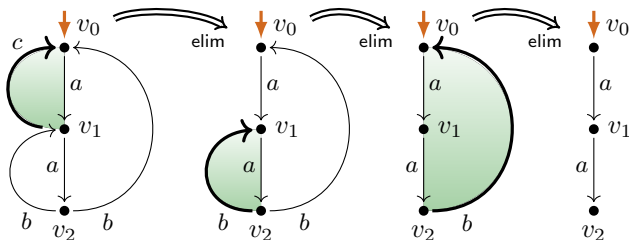
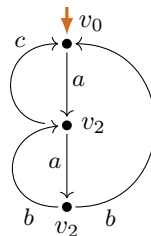


not LEE

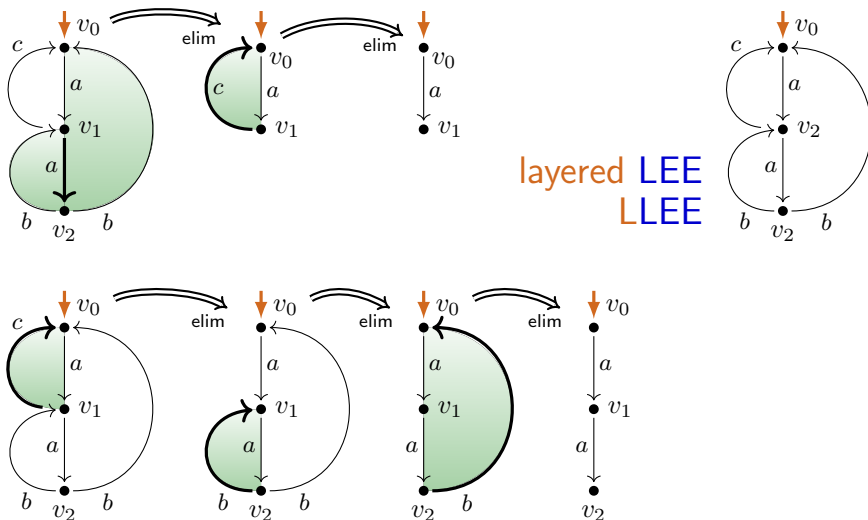
LEE



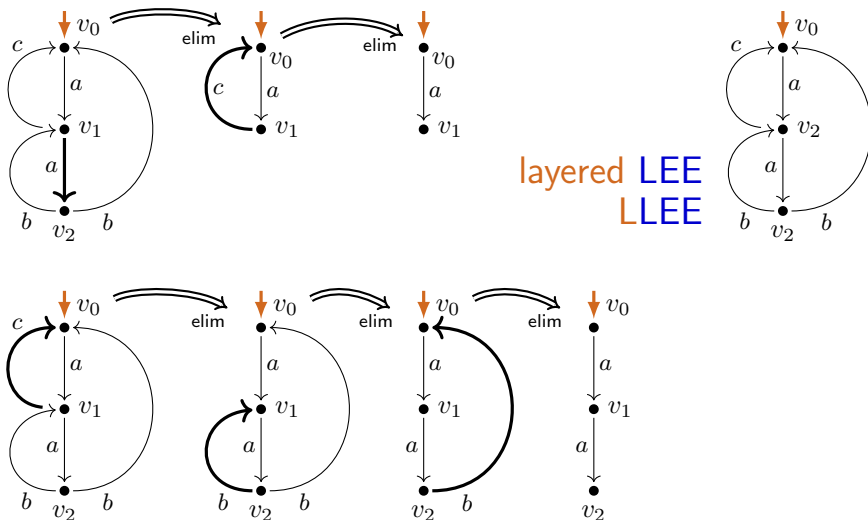
LEE



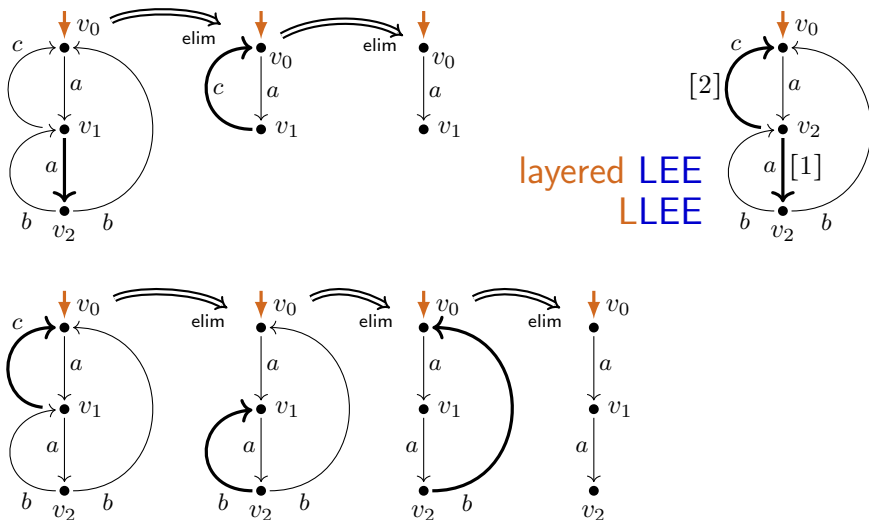
Layered LEE



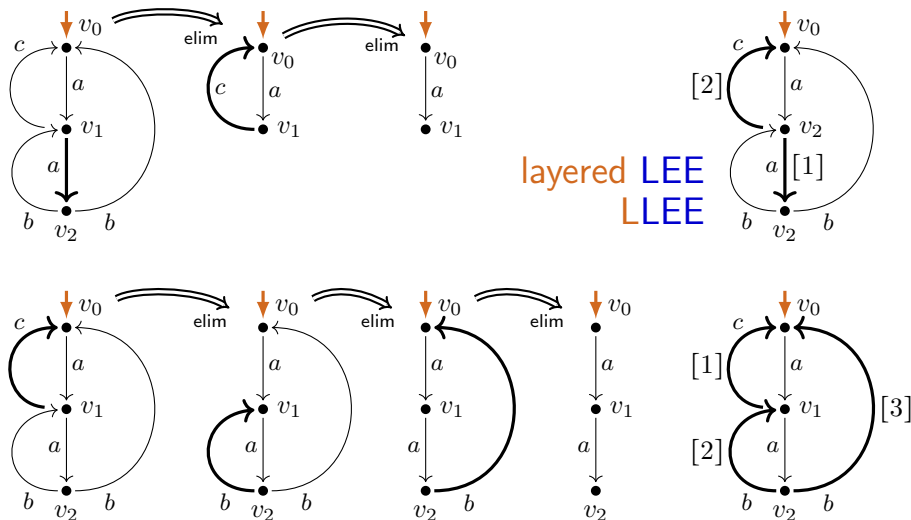
Layered LEE



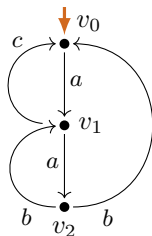
Layered LEE



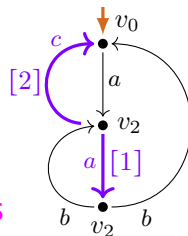
LLEE-witness and LLEE-graph



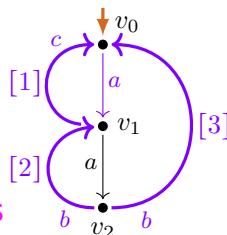
LLEE-witness and LLEE-graph



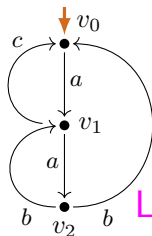
LLEE-witness



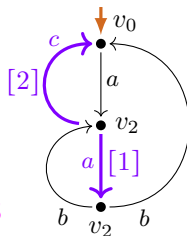
LLEE-witness



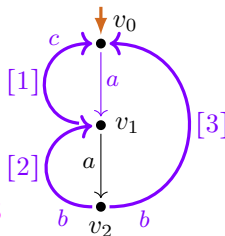
LLEE-witness and LLEE-graph



LLEE-graph

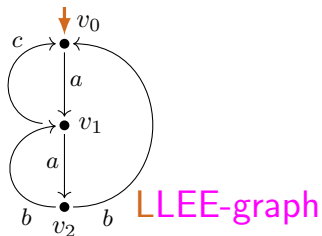


LLEE-witness

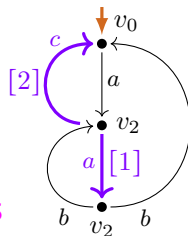


LLEE-witness

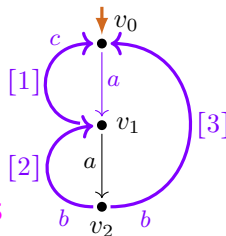
LLEE-witness and LLEE-graph



layered
LLEE-witness

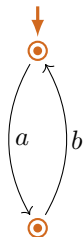


layered
LLEE-witness



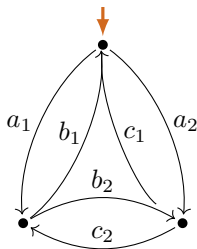
LLEE versus P -/ $[\![\cdot]\!]$ P -expressibility (examples)

G_1

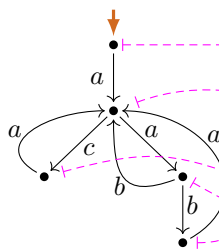


not P -expressible
not $[\![\cdot]\!]$ P -expressible

G_2

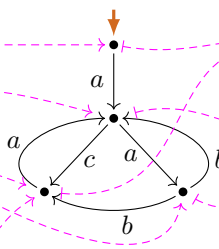


$/ G_3$



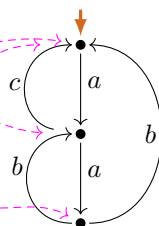
P -expressible?
 $[\![\cdot]\!]$ P -expressible

G_4



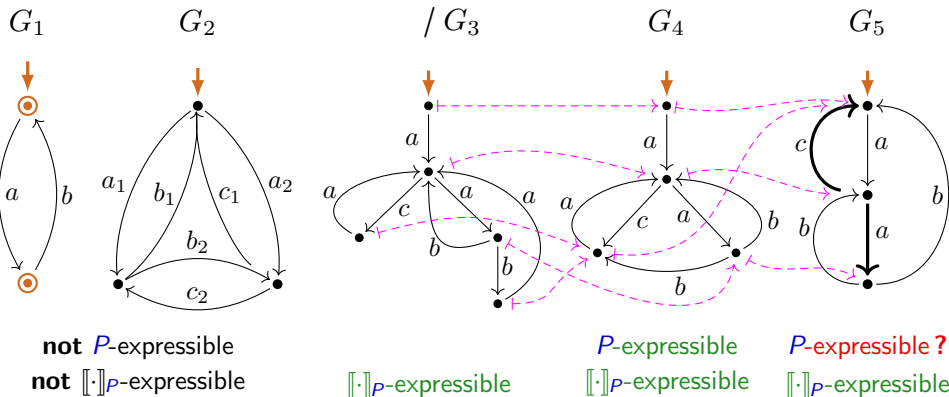
P -expressible
 $[\![\cdot]\!]$ P -expressible

G_5



P -expressible?
 $[\![\cdot]\!]$ P -expressible

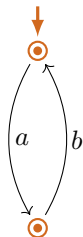
LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)



LLEE

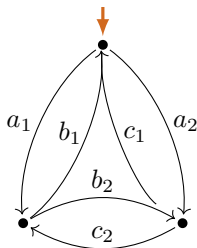
LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

G_1

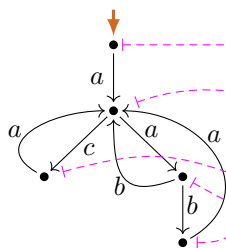


not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

G_2

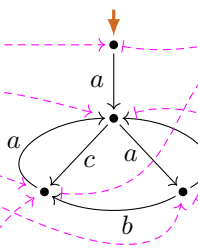


$/ G_3$



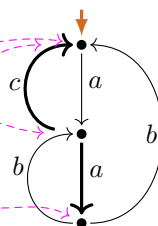
$\llbracket \cdot \rrbracket_P$ -expressible

G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

G_5



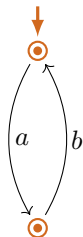
P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE

1-LLEE

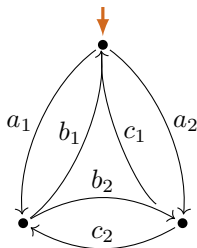
LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

G_1

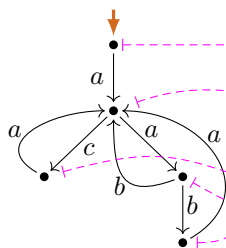


not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

G_2

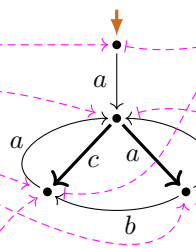


$/ G_3$



$\llbracket \cdot \rrbracket_P$ -expressible

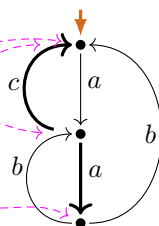
G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE

G_5

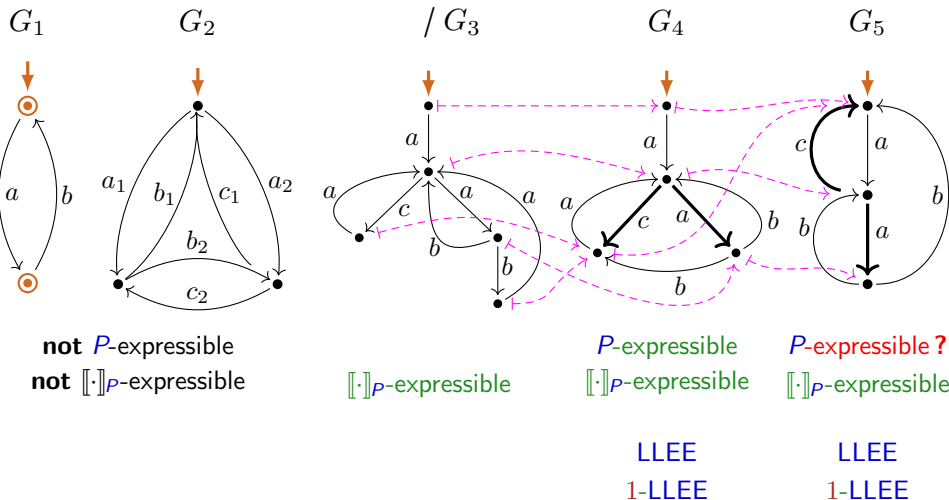


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE

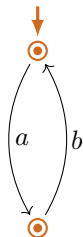
1-LLEE

LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)



LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

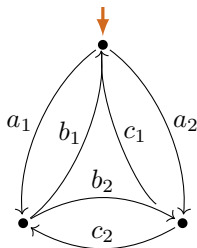
G_1



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

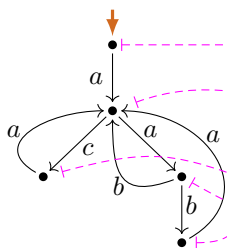
not LLEE

G_2

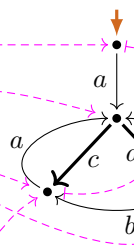


$\llbracket \cdot \rrbracket_P$ -expressible

$/ G_3$



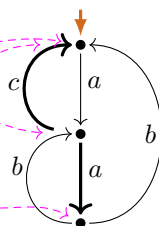
G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

G_5

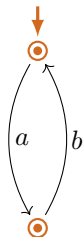


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

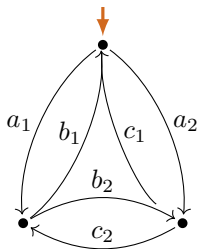
G_1



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

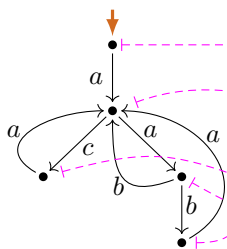
not LLEE
not 1-LLEE

G_2

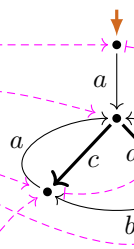


$\llbracket \cdot \rrbracket_P$ -expressible

$/ G_3$



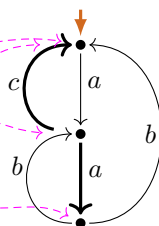
G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

G_5

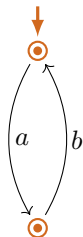


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

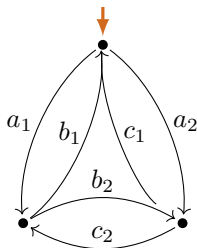
G_1



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE
not 1-LLEE

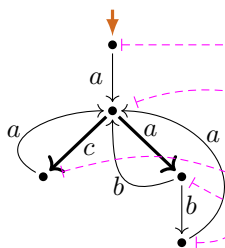
G_2



$\llbracket \cdot \rrbracket_P$ -expressible

LLEE

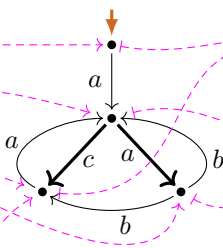
$/ G_3$



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

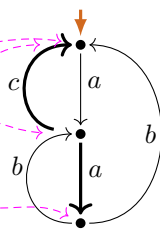
G_4



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

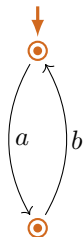
LLEE
1-LLEE

G_5



LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

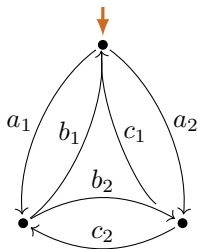
G_1



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE
not 1-LLEE

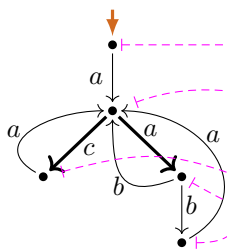
G_2



$\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

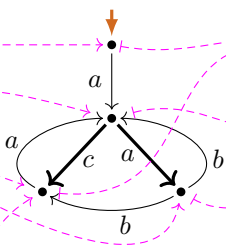
$/ G_3$



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

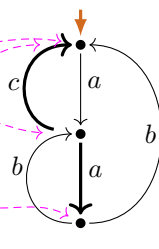
G_4



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

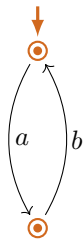
LLEE
1-LLEE

G_5



LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

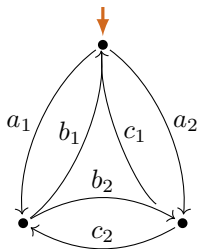
G_1



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE
not 1-LLEE

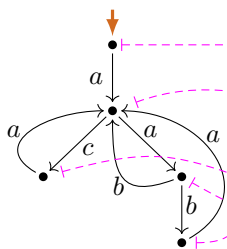
G_2



$\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

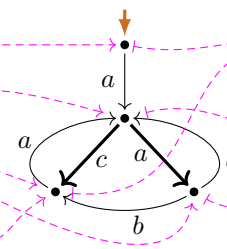
$/ G_3$



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

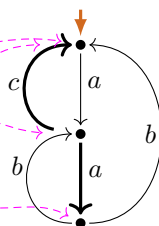
G_4



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

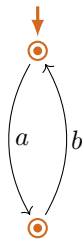
LLEE
1-LLEE

G_5



LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

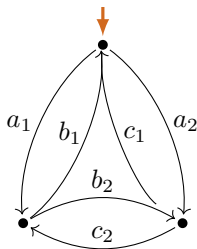
G_1



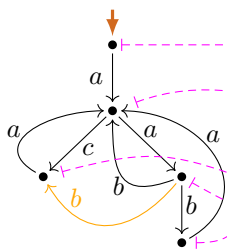
not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE
not 1-LLEE

G_2



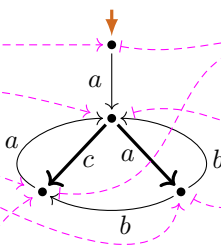
G_3^+ / G_3



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

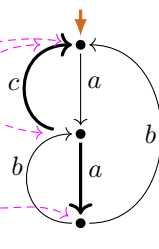
G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

G_5

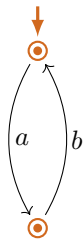


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

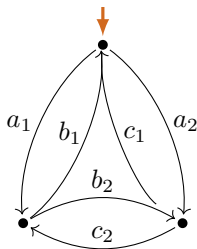
G_1



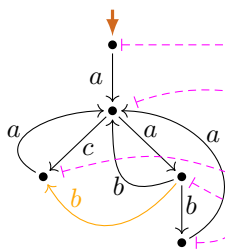
not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE
not 1-LLEE

G_2



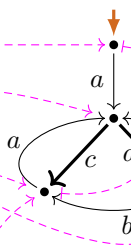
G_3^+ / G_3



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE / LLEE
1-LLEE

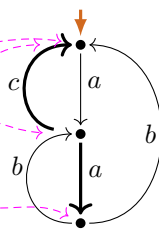
G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

G_5

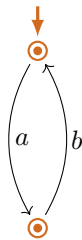


P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

LLEE versus P -/ $[\cdot]$ P -expressibility (examples)

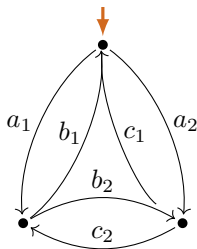
G_1



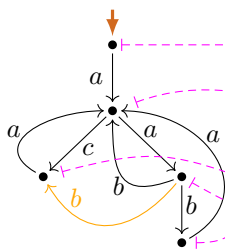
not P -expressible
not $[\cdot]$ P -expressible

not LLEE
not 1-LLEE

G_2



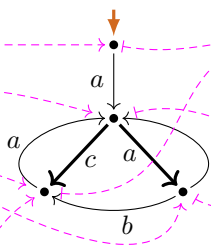
G_3^+ / G_3



P -expressible?
 $[\cdot]$ P -expressible

not LLEE / LLEE
not 1-LLEE / 1-LLEE

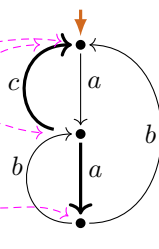
G_4



P -expressible
 $[\cdot]$ P -expressible

LLEE
1-LLEE

G_5

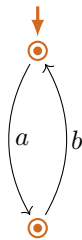


P -expressible?
 $[\cdot]$ P -expressible

LLEE
1-LLEE

LLEE versus P -/ $\llbracket \cdot \rrbracket_P$ -expressibility (examples)

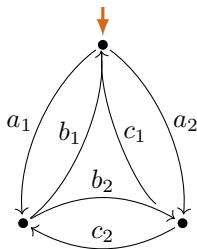
G_1



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE
not 1-LLEE

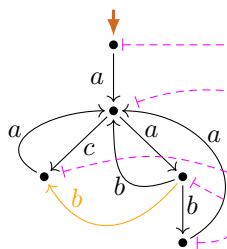
G_2



not P -expressible
not $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE / LLEE
not 1-LLEE / 1-LLEE

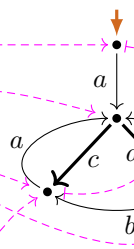
G_3^+ / G_3



$P(\cdot)$ -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

not LLEE / LLEE
not 1-LLEE / 1-LLEE

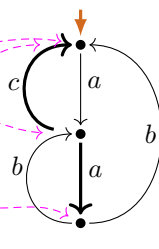
G_4



P -expressible
 $\llbracket \cdot \rrbracket_P$ -expressible

LLEE
1-LLEE

G_5



P -expressible?
 $\llbracket \cdot \rrbracket_P$ -expressible

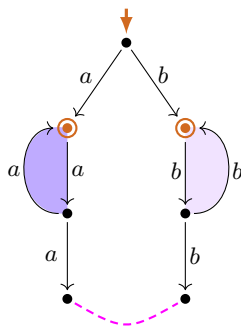
LLEE
1-LLEE

TERMGRAPH '24

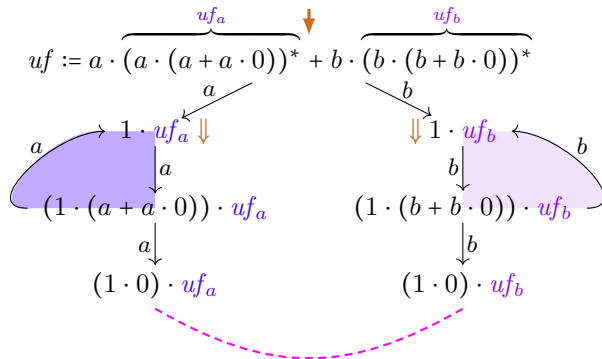
- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ characterisation of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ characterization of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Image of P is **not** closed under bisimulation collapse

$P(uf)$



$P(uf)$



Process interpretation P (formally)

Definition (Transition system specification $\mathcal{T}$)

$$\begin{array}{c}
 \frac{}{1 \Downarrow} \qquad \frac{e_i \Downarrow}{(e_1 + e_2) \Downarrow} \ (i \in \{1, 2\}) \qquad \frac{e_1 \Downarrow \quad e_2 \Downarrow}{(e_1 \cdot e_2) \Downarrow} \qquad \frac{}{(e^*) \Downarrow} \\
 \\
 \frac{}{a \xrightarrow{a} 1} \qquad \frac{e_i \xrightarrow{a} e'_i}{e_1 + e_2 \xrightarrow{a} e'_i} \ (i \in \{1, 2\}) \\
 \\
 \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \qquad \frac{e_1 \Downarrow \quad e_2 \xrightarrow{a} e'_2}{e_1 \cdot e_2 \xrightarrow{a} e'_2} \qquad \frac{e \xrightarrow{a} e'}{e^* \xrightarrow{a} e' \cdot e^*}
 \end{array}$$

Responsible rule

Definition (Transition system specification $\mathcal{T}$)

$$\frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2}$$

Problem: if e'_1 is **not** normed, $e'_1 \cdot e_2$ has **redundant** subexpression e_2

Compact process interpretation $P^\bullet$

Definition (Transition system specification $\mathcal{T}^\bullet$, changed rules w.r.t. $\mathcal{T}$)

$$\frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1} \text{ (if } e'_1 \text{ is not normed)}$$

Compact process interpretation $P^\bullet$

Definition (Transition system specification $\mathcal{T}^\bullet$, changed rules w.r.t. $\mathcal{T}$)

$$\frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1 \cdot e_2} \text{ (if } e'_1 \text{ is normed)} \qquad \frac{e_1 \xrightarrow{a} e'_1}{e_1 \cdot e_2 \xrightarrow{a} e'_1} \text{ (if } e'_1 \text{ is not normed)}$$

Compact process interpretation $P^\bullet$

Definition (Transition system specification $\mathcal{T}^\bullet$, changed rules w.r.t. $\mathcal{T}$)

$$\pi(e) := \begin{cases} e_1 \cdot e_2 & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is normed,} \\ \pi(e_1) & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is not normed,} \\ e & \text{otherwise.} \end{cases}$$

$$\frac{e_1 \xrightarrow{a} ed}{e_1 \cdot e_2 \xrightarrow{a} \pi(ed \cdot e_2)}$$

Compact process interpretation $P^\bullet$

Definition (Transition system specification $\mathcal{T}^\bullet$, changed rules w.r.t. $\mathcal{T}$)

$$\pi(e) := \begin{cases} e_1 \cdot e_2 & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is normed,} \\ \pi(e_1) & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is not normed,} \\ e & \text{otherwise.} \end{cases}$$

$$\frac{e_1 \xrightarrow{a} ed}{e_1 \cdot e_2 \xrightarrow{a} \pi(ed \cdot e_2)}$$

$$\frac{e \xrightarrow{a} ed}{e^* \xrightarrow{a} \pi(ed \cdot e^*)}$$

Compact process interpretation $P^\bullet$

Definition (Transition system specification $\mathcal{T}^\bullet$, changed rules w.r.t. $\mathcal{T}$)

$$\pi(e) := \begin{cases} e_1 \cdot e_2 & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is \textcolor{brown}{normed}}, \\ \pi(e_1) & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is \textcolor{red}{not normed}}, \\ e & \text{otherwise.} \end{cases}$$

$$\frac{e_1 \xrightarrow{a} ed}{e_1 \cdot e_2 \xrightarrow{a} \pi(ed \cdot e_2)}$$

$$\frac{e \xrightarrow{a} ed}{e^* \xrightarrow{a} \pi(ed \cdot e^*)}$$

Definition

The **compact process (graph) interpretation** $P^\bullet(e)$ of a reg. expr's e :

$P^\bullet(e) :=$ **labeled transition graph** generated by $\pi(e)$ by derivations in $\mathcal{T}^\bullet$.

Compact process interpretation $P^\bullet$

Definition (Transition system specification $\mathcal{T}^\bullet$, changed rules w.r.t. $\mathcal{T}$)

$$\pi(e) := \begin{cases} e_1 \cdot e_2 & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is \textcolor{brown}{normed}}, \\ \pi(e_1) & \text{if } e = e_1 \cdot e_2 \text{ and } e_1 \text{ is \textcolor{red}{not normed}}, \\ e & \text{otherwise.} \end{cases}$$

$$\frac{e_1 \xrightarrow{a} ed}{e_1 \cdot e_2 \xrightarrow{a} \pi(ed \cdot e_2)}$$

$$\frac{e \xrightarrow{a} ed}{e^* \xrightarrow{a} \pi(ed \cdot e^*)}$$

Definition

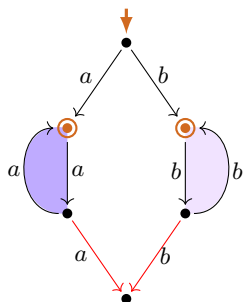
The **compact process (graph) interpretation** $P^\bullet(e)$ of a reg. expr's e :
 $P^\bullet(e) :=$ **labeled transition graph** generated by $\pi(e)$ by derivations in $\mathcal{T}^\bullet$.

Lemma ($P^\bullet$ increases sharing; $P^\bullet, P$ have same bisimulation semantics)

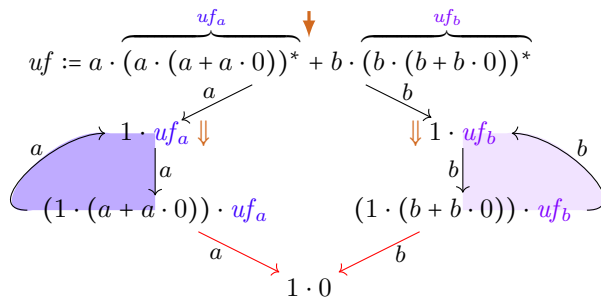
- (i) $P(e) \Rightarrow P^\bullet(\pi(e))$ for all regular expressions e .
- (ii) (G is $\llbracket \cdot \rrbracket_{P^\bullet}$ -expressible $\iff G$ is $\llbracket \cdot \rrbracket_P$ -expressible) for all graphs G .

Image of $P^\bullet$ under bisimulation collapse ...

$P^\bullet(uf)$



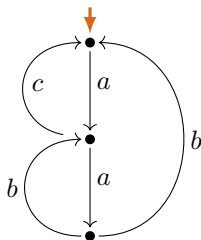
$P^\bullet(uf)$



Refined extraction expression (example)

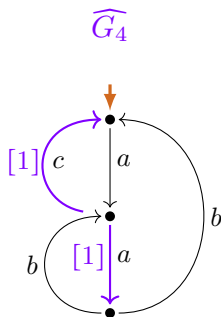
Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!

G_4



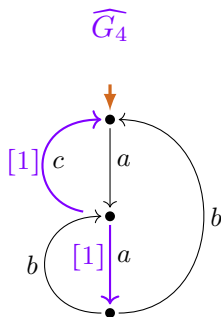
Refined extraction expression (example)

Define readback steps to precisely reverse $P^\bullet$ -defined transitions!



Refined extraction expression (example)

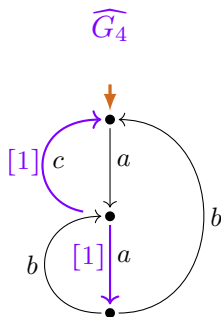
Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!



$$(1 \cdot (\quad)^*) \cdot 0$$

Refined extraction expression (example)

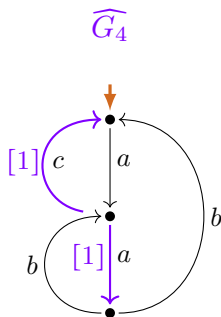
Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!



$$(1 \cdot (\begin{array}{c} \vdots \\ a \\ \vee \end{array})^*) \cdot 0$$

Refined extraction expression (example)

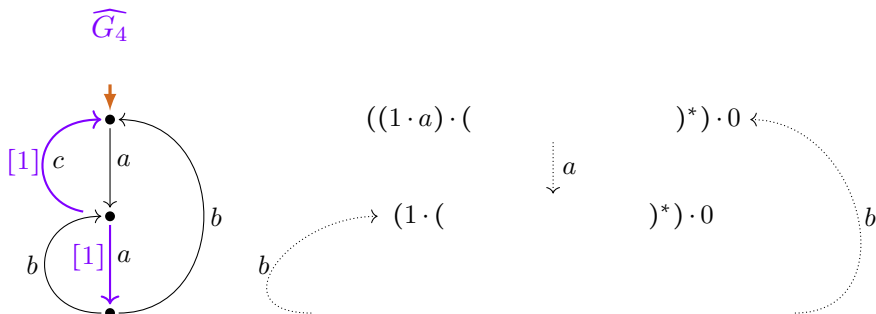
Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!



$$\begin{array}{c}
 ((1 \cdot a) \cdot (\quad)^*) \cdot 0 \\
 \downarrow a \\
 (1 \cdot (\quad)^*) \cdot 0
 \end{array}$$

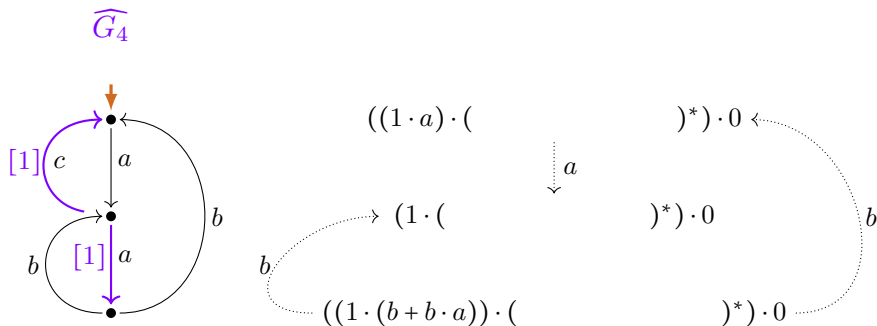
Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!



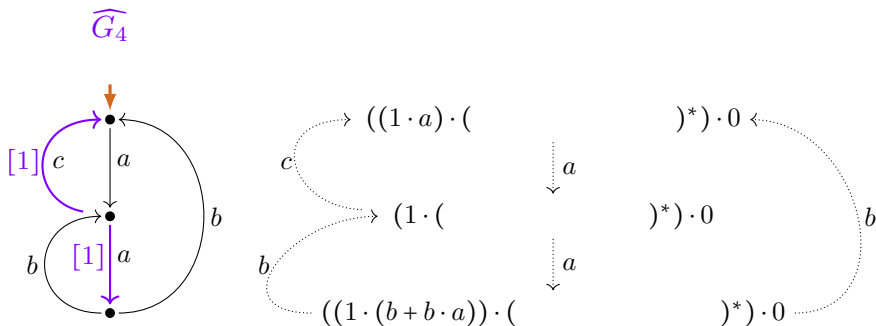
Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!



Refined extraction expression (example)

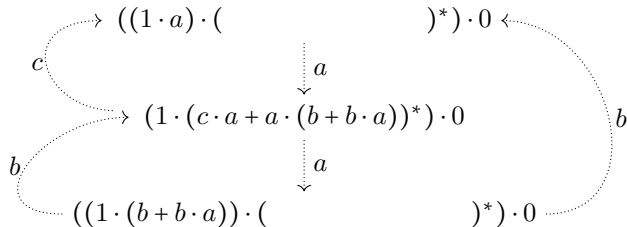
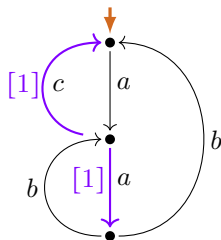
Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!



Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!

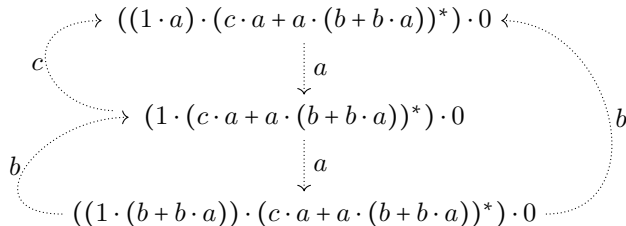
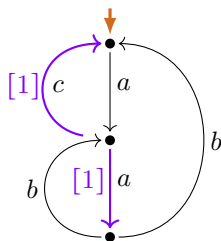
$\widehat{G}_4$



Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!

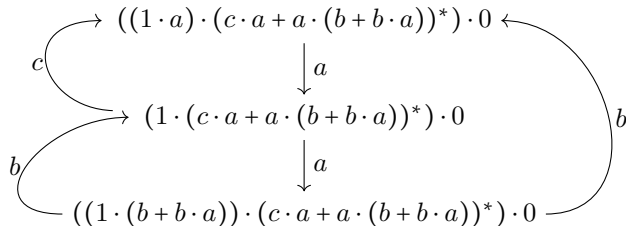
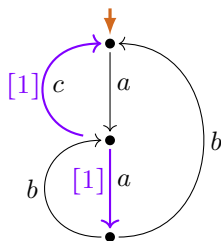
$\widehat{G}_4$



Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!

$\widehat{G}_4$

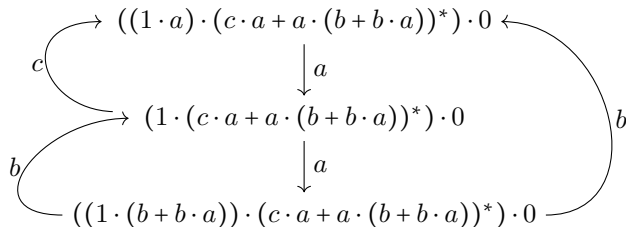
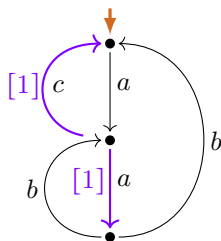


Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!

$\widehat{G}_4$

$$P^\bullet(uf) = P(uf) \simeq G_4$$

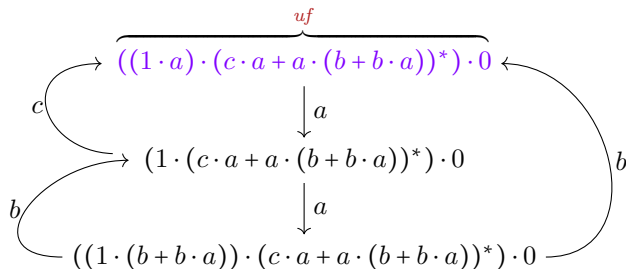
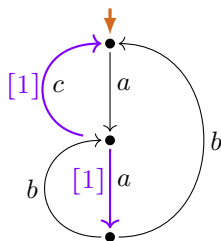


Refined extraction expression (example)

Define **readback steps** to **precisely reverse** $P^\bullet$ -defined transitions!

$\widehat{G}_4$

$$P^\bullet(uf) = P(uf) \simeq G_4$$



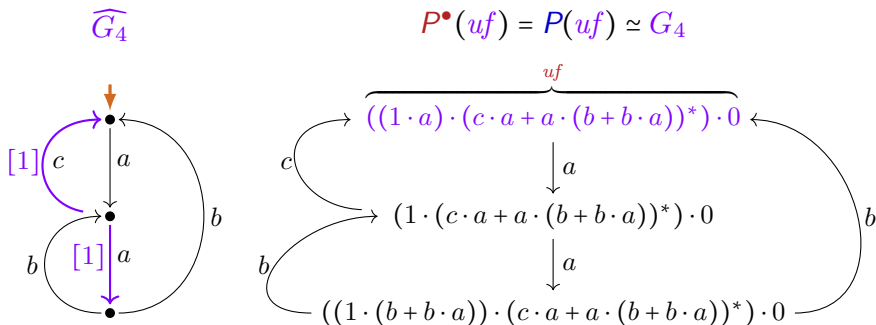
Refined extraction expression (example)

Lemma (*TERMGRAPH '24*)

Process graphs with LLEE are $P^\bullet$ -expressible by $(*/\perp)$ reg. expressions:

$$\text{LLEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} (G \rightrightarrows P^\bullet(uf)).$$

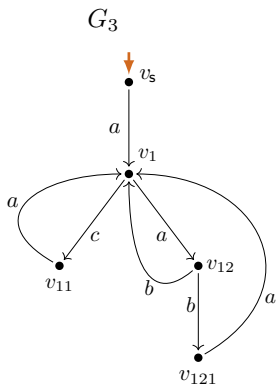
Proof: Define readback steps to precisely reverse $P^\bullet$ -defined transitions!



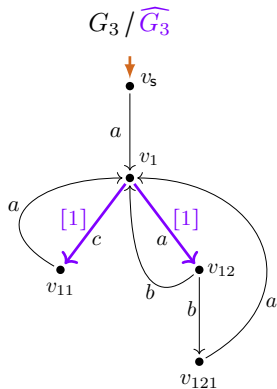
Characterising the image of $P^\bullet$ for $(*/\perp)$ expressions

- ▶ Regular expressions (unary/binary star, 1-free/[under-star-1-free](#) $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility question](#))
- ▶ Layered Loop existence and elimination ([LLEE](#)/[1-LLEE](#))
 - ▶ interpretation/[extraction](#) correspondences with $(*/\perp)$ /[all](#) reg. expr.
 - ▶ [LLEE](#) is preserved by bisimulation collapse ([1-LLEE](#) is [not](#))
- ▶ TERMGRAPH 2024: [compact](#) process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is [not](#))
- ▶ [characterisation](#) of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ [characterization](#) of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Refined extraction

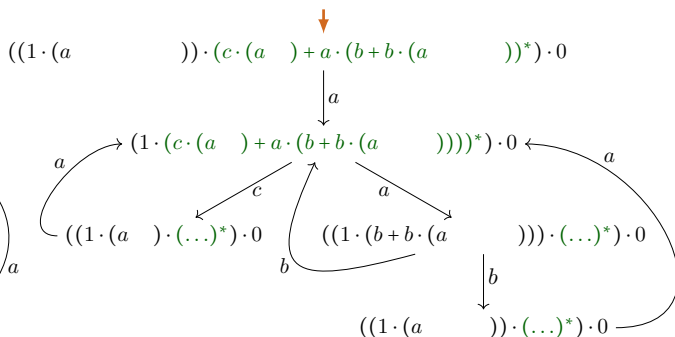
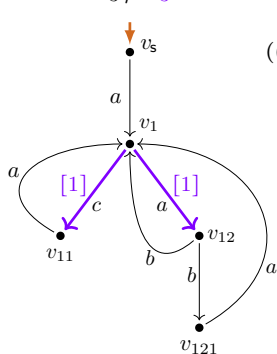


Refined extraction

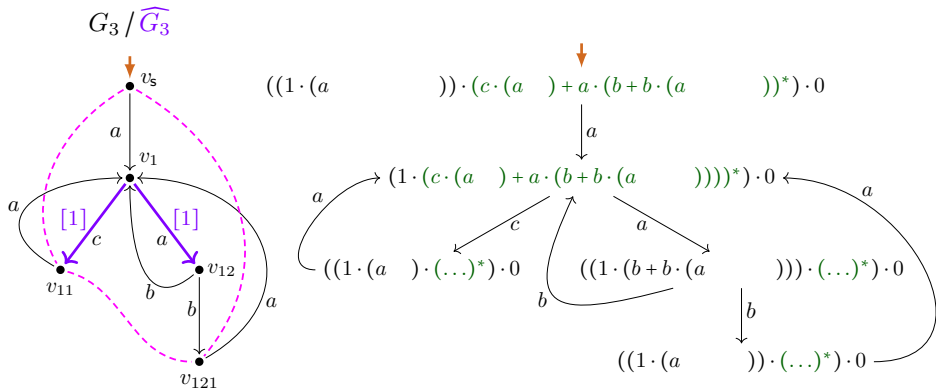


Refined extraction

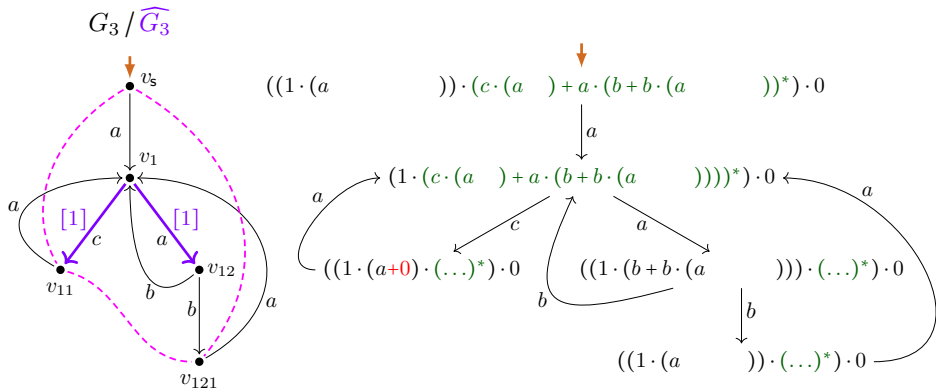
$G_3 / \widehat{G}_3$



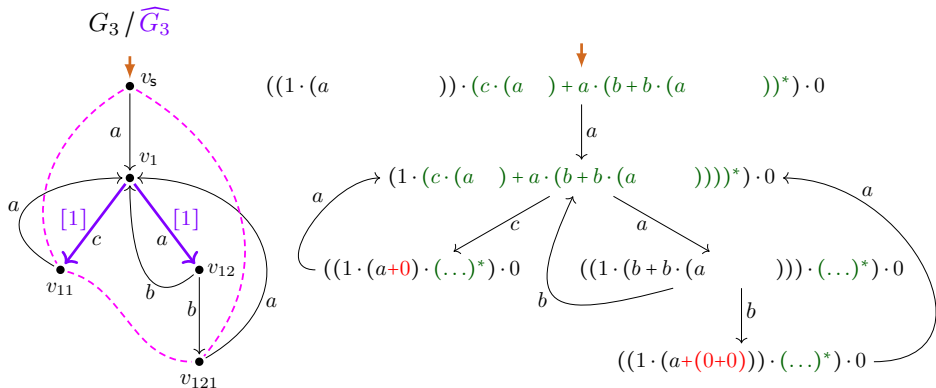
Refined extraction



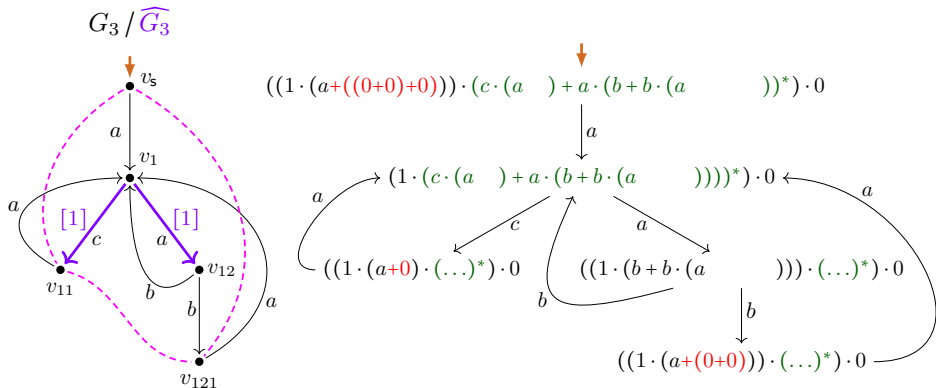
Refined extraction, adapted to prevent sharing



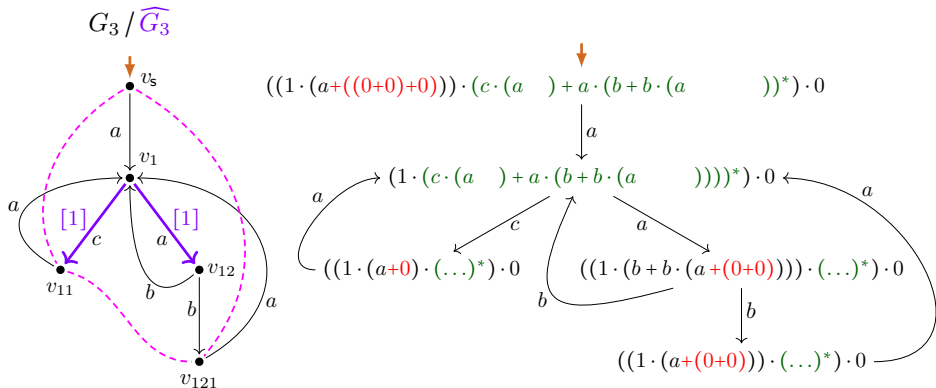
Refined extraction, adapted to prevent sharing



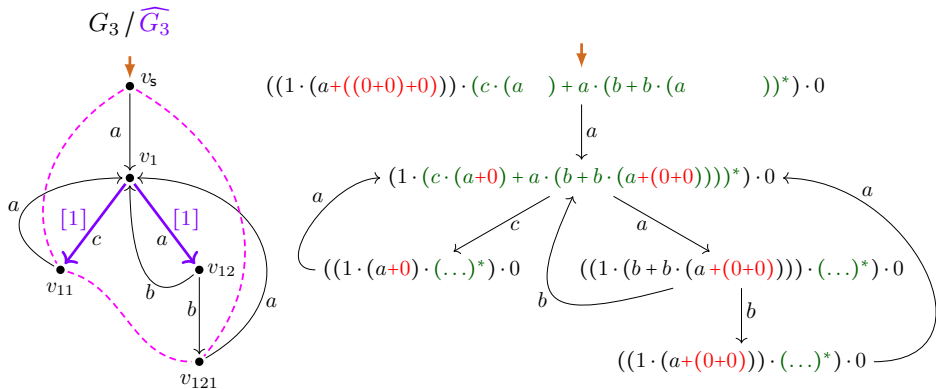
Refined extraction, adapted to prevent sharing



Refined extraction, adapted to prevent sharing

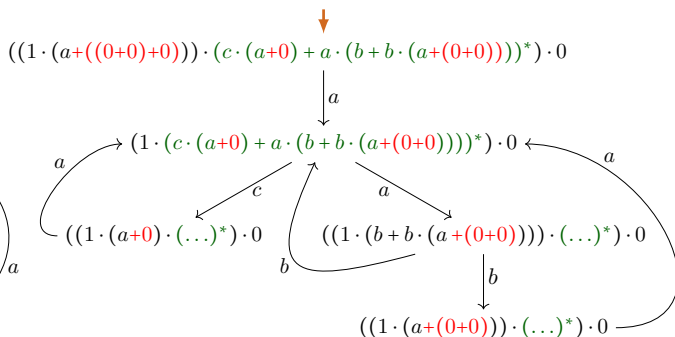
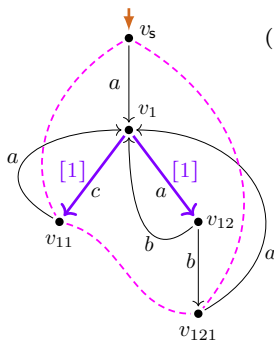


Refined extraction, adapted to prevent sharing



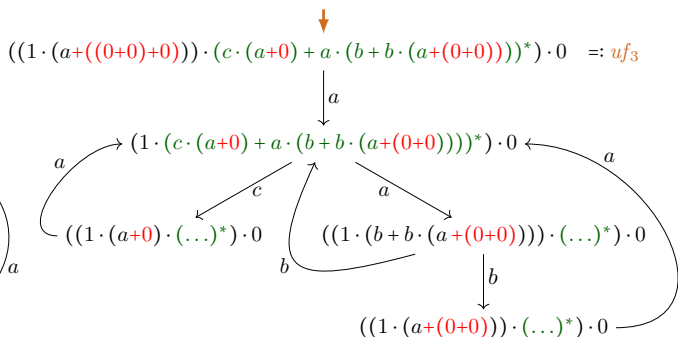
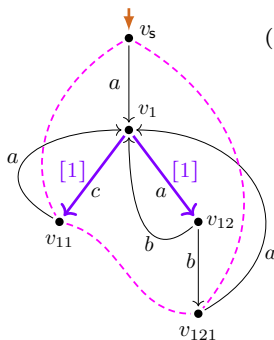
Refined extraction, adapted to prevent sharing

$G_3 / \widehat{G}_3$

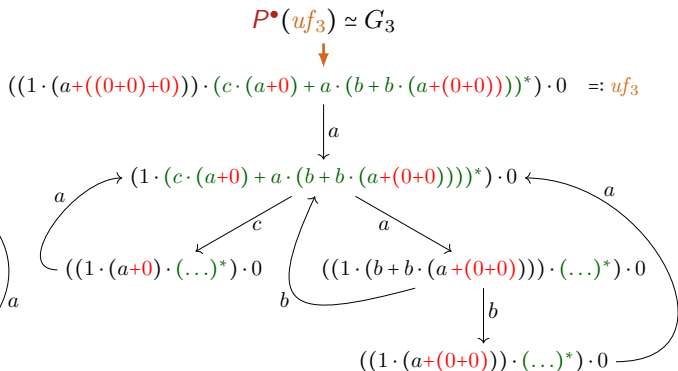
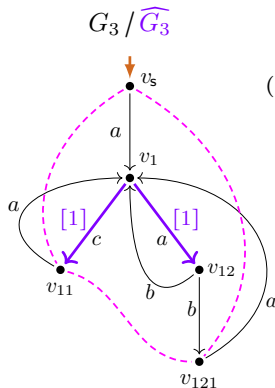


Refined extraction, adapted to prevent sharing

$G_3 / \widehat{G}_3$



Refined extraction, adapted to prevent sharing



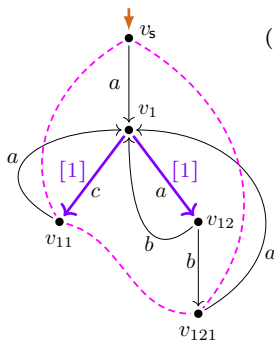
Refined extraction, adapted to prevent sharing

Lemma [Refined extraction, prevent sharing]

From every graph G with LLEE

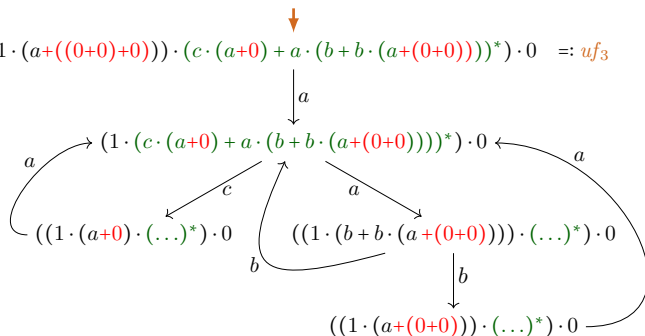
a $(*/\perp)$ regular expression uf can be extracted such that $G \simeq P^\bullet(uf)$.

$G_3 / \widehat{G}_3$



$P^\bullet(uf_3) \simeq G_3$

$$((1 \cdot (a + ((0+0)+0))) \cdot (c \cdot (a+0) + a \cdot (b + b \cdot (a + (0+0))))^*) \cdot 0 =: uf_3$$



Characterizing the image of $P^\bullet$ for $(*/\perp)$ expressions

Lemma [Refined extraction, prevent sharing]

From every graph G with LLEE

a $(*/\perp)$ regular expression uf can be extracted such that $G \simeq P^\bullet(uf)$.

Characterizing the image of $P^\bullet$ for $(*/\perp)$ expressions

Lemma [Refined extraction, prevent sharing]

From every graph G with LLEE

a $(*/\perp)$ regular expression uf can be extracted such that $G \simeq P^\bullet(uf)$.

Statement

A process graph G is $P^\bullet$ -expressible by an $(*/\perp)$ regular expression

$\iff G$ is finite, and G satisfies LLEE.

Characterizing the image of $P^\bullet$ for $(*/\perp)$ expressions

Lemma [Refined extraction, prevent sharing]

From every graph G with LLEE

a $(*/\perp)$ regular expression uf can be extracted such that $G \simeq P^\bullet(uf)$.

Statement

A process graph G is $P^\bullet$ -expressible by an $(*/\perp)$ regular expression

$\iff G$ is finite, and G satisfies LLEE.

That is: $\text{im}(P^\bullet) \upharpoonright_{\text{RExp}(*/\perp)} = \{G \mid G \text{ is finite and satisfies LLEE}\}$.

Consequence

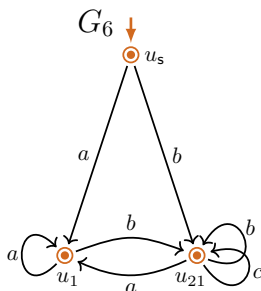
For every process graph G , TFAE:

- (i) G is $\llbracket \cdot \rrbracket_P$ -expressible by an $(*/\perp)$ regular expression.
- (ii) The bisimulation collapse of G is finite, and satisfies LLEE.
- (iii) The bisimulation collapse of G
is $P^\bullet$ -expressible by a $(*/\perp)$ regular expression.

Characterising the image of $P^\bullet$

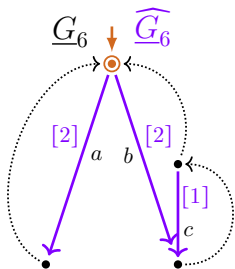
- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ characterisation of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ characterization of image of $P^\bullet$
- ▶ Relationship with co-reducibility

Refined extraction from 1-LLEE graph

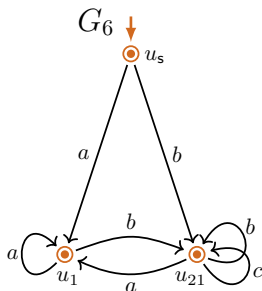


$\neg\text{LLEE}((\underline{G_6}])$

Refined extraction from 1-LLEE graph

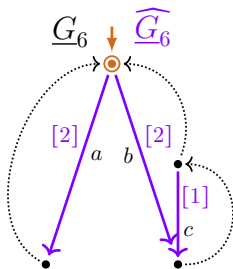


$\text{LLEE}(\underline{G}_6)$

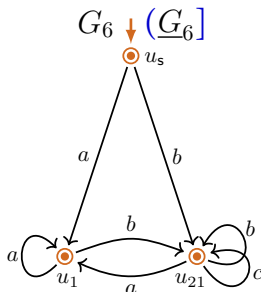


$\neg \text{LLEE}((\underline{G}_6])$

Refined extraction from 1-LLEE graph

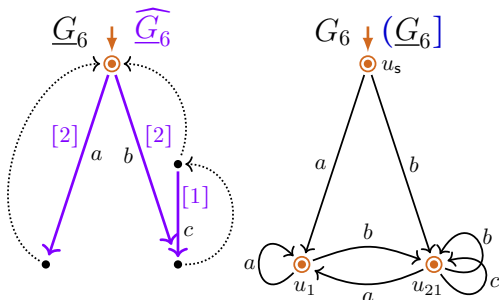


$LLEE(\underline{G}_6)$



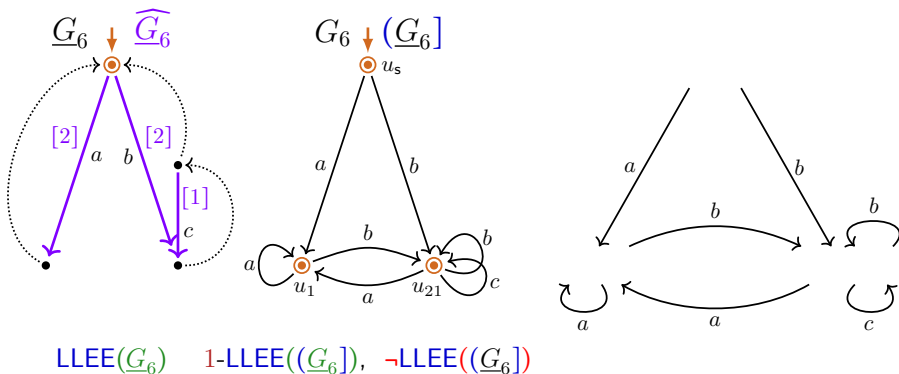
$\neg LLEE((\underline{G}_6])$

Refined extraction from 1-LLEE graph

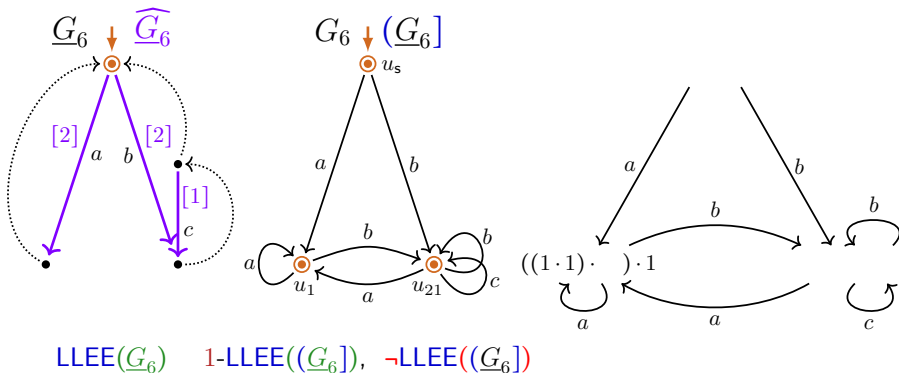


$$\text{LLEE}(\underline{G}_6) \quad 1\text{-LLEE}((\underline{G}_6]), \neg\text{LLEE}((\underline{G}_6])$$

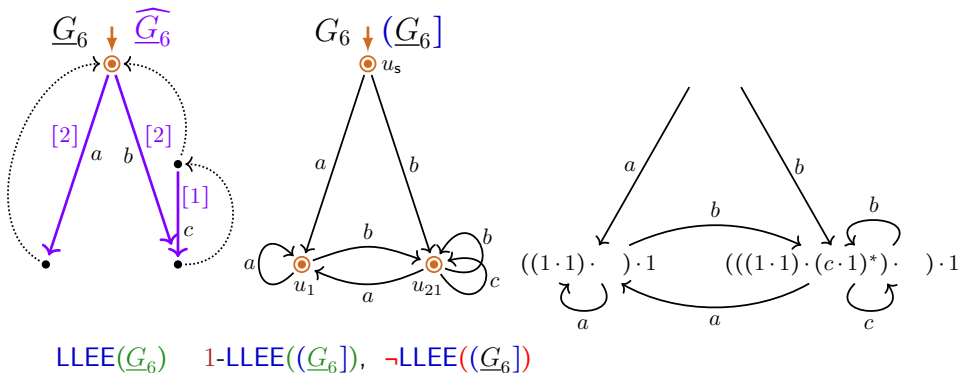
Refined extraction from 1-LLEE graph



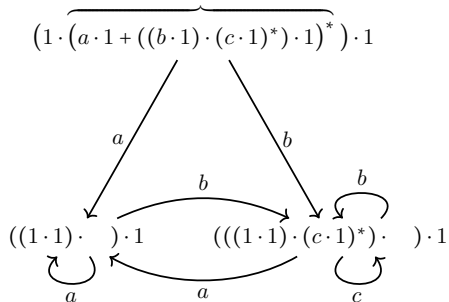
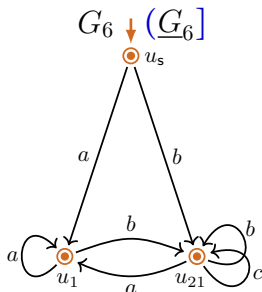
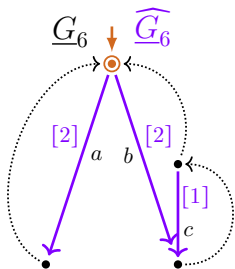
Refined extraction from 1-LLEE graph



Refined extraction from 1-LLEE graph

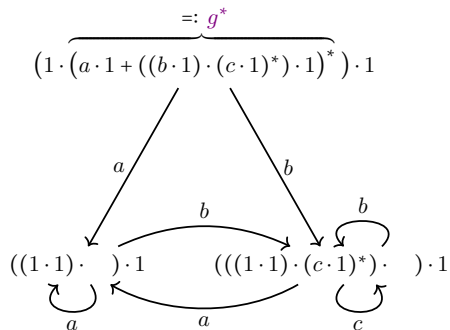
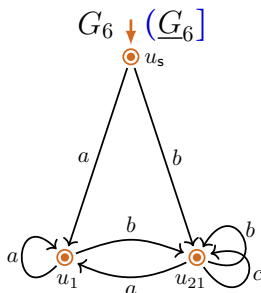
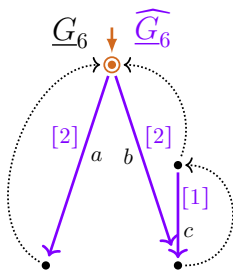


Refined extraction from 1-LLEE graph



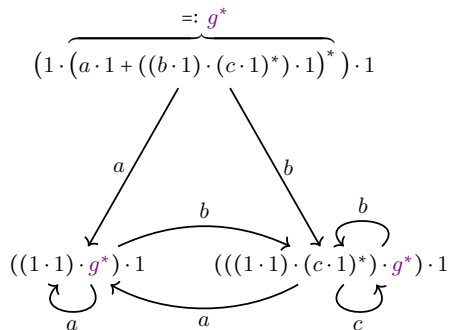
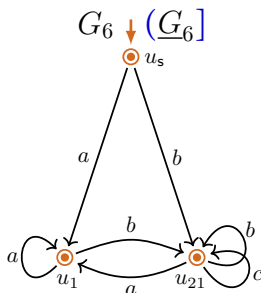
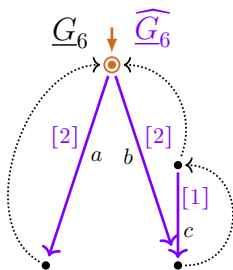
$$\text{LLEE}(\underline{G_6}) \quad 1\text{-LLEE}((\underline{G_6}]), \quad \neg\text{LLEE}((\underline{G_6}])$$

Refined extraction from 1-LLEE graph



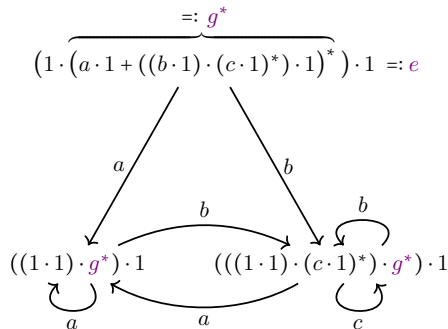
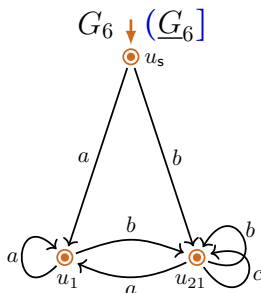
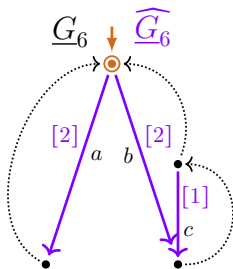
$LLEE(\underline{G_6})$ $1-LLEE([G_6])$, $\neg LLEE([G_6])$

Refined extraction from 1-LLEE graph



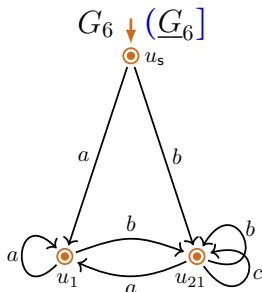
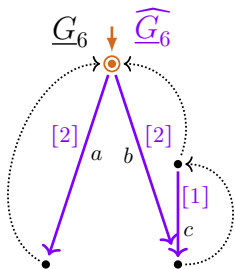
$LLEE(\underline{G_6})$ $1-LLEE((\underline{G_6}])$, $\neg LLEE((\underline{G_6}])$

Refined extraction from 1-LLEE graph

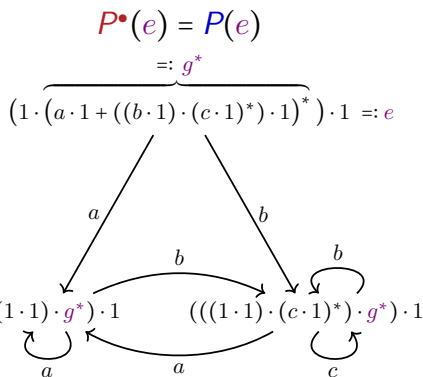


$LLEE(\underline{G_6})$ $1-LLEE((\underline{G_6})), \neg LLEE((\underline{G_6}))$

Refined extraction from 1-LLEE graph



$LLEE(\underline{G}_6)$ $1-LLEE((\underline{G}_6]), \neg LLEE((\underline{G}_6])$



Characterizing the image of $P^\bullet$

Lemma [Refined extraction, prevent sharing]

From every graph G with 1-LLEE

a regular expression e can be extracted such that $G \simeq P^\bullet(e)$.

Characterizing the image of $P^\bullet$

Lemma [Refined extraction, prevent sharing]

From every graph G with 1-LLEE
a regular expression e can be extracted such that $G \simeq P^\bullet(e)$.

Lemma (G , LICS 2024)

$P(e)$ satisfies 1-LLEE, for all regular expressions.

Holds also for $P^\bullet(e)$.

Characterizing the image of $P^\bullet$

Lemma [Refined extraction, prevent sharing]

From every graph G with 1-LLEE
a regular expression e can be extracted such that $G \simeq P^\bullet(e)$.

Lemma (G , LICS 2024)

$P(e)$ satisfies 1-LLEE, for all regular expressions.

Holds also for $P^\bullet(e)$.

Statement

A process graph G is $P^\bullet$ -expressible by a regular expression
 $\iff G$ is finite, and G satisfies 1-LLEE.

Characterizing the image of $P^\bullet$

Lemma [Refined extraction, prevent sharing]

From every graph G with 1-LLEE
a regular expression e can be extracted such that $G \simeq P^\bullet(e)$.

Lemma (G , LICS 2024)

$P(e)$ satisfies 1-LLEE, for all regular expressions.

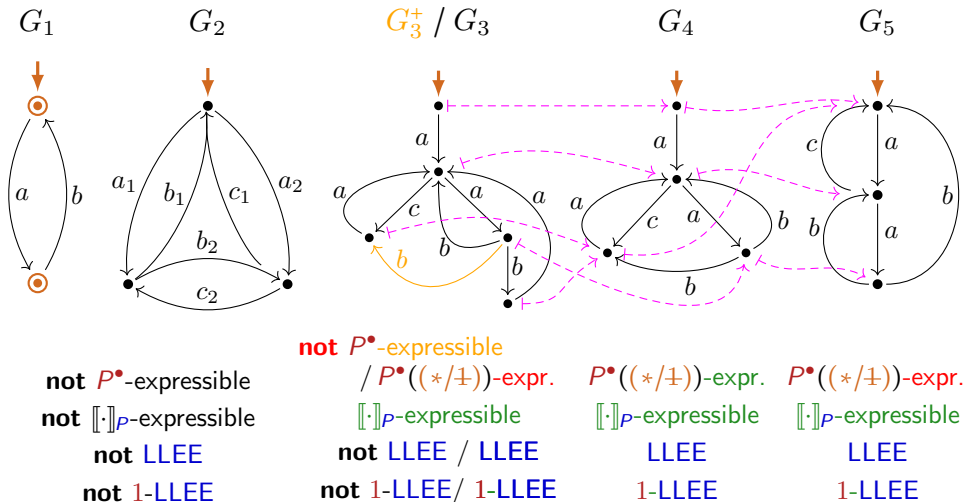
Holds also for $P^\bullet(e)$.

Statement

A process graph G is $P^\bullet$ -expressible by a regular expression
 $\iff G$ is finite, and G satisfies 1-LLEE.

That is: $\text{im}(P^\bullet) = \{G \mid G \text{ is finite and satisfies 1-LLEE}\}$
 $= \{(\underline{G}) \mid \text{1-graph } \underline{G} \text{ is finite and satisfies LLEE}\}.$

$P^\bullet$ -expressibility (final results)



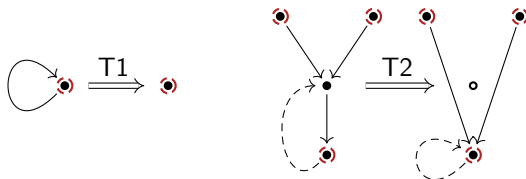
Relationship with Co-Reducibility

- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ characterisation of image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ characterization of image of $P^\bullet$
- ▶ Relationship with co-reducibility

LLEE = co-reducible⁺ to acyclic graph

Connection to work by Laarakker and Kappé (TERMGRAPH 2026) on:

- ▶ co-reducibility of state graphs,
- ▶ based on flow graph reducibility by Hecht and Ullman (1972).

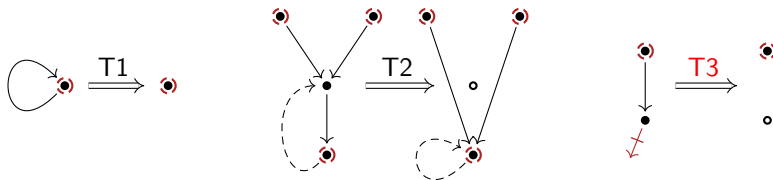


LLEE = co-reducible⁺ to acyclic graph

Connection to work by Laarakker and Kappé (TERMGRAPH 2026) on:

- ▶ co-reducibility of state graphs,
- ▶ based on flow graph reducibility by Hecht and Ullman (1972).

We extend co-reducibility to co-reducibility⁺ by adding a prune operation:

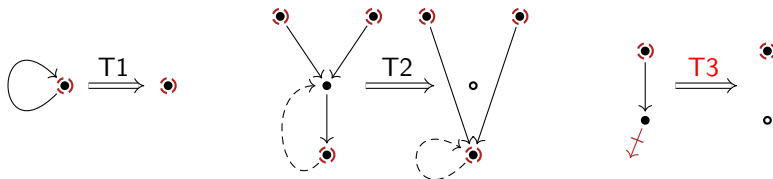


LLEE = co-reducible⁺ to acyclic graph

Connection to work by Laarakker and Kappé (TERMGRAPH 2026) on:

- ▶ co-reducibility of state graphs,
- ▶ based on flow graph reducibility by Hecht and Ullman (1972).

We extend co-reducibility to co-reducibility⁺ by adding a prune operation:



Statement

A process graph G has the property LLEE

$\iff$ the state graph of G is co-reducible⁺ to an acyclic graph.

Summary

- ▶ Regular expressions (unary/binary star, 1-free/[under-star-1-free](#) ($*/\perp$))
- ▶ Milner's process interpretation P / semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ [expressibility question](#))
- ▶ Layered Loop existence and elimination ([LLEE](#)/[1-LLEE](#))
 - ▶ interpretation/[extraction](#) correspondences with ($*/\perp$)/all reg. expr.
 - ▶ [LLEE](#) is preserved by bisimulation collapse ([1-LLEE](#) is [not](#))
- ▶ TERMGRAPH 2024: [compact](#) process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is [not](#))
- ▶ [LLEE](#) $\triangleq$ image of $P^\bullet$ restricted to ($*/\perp$) expressions
- ▶ [1-LLEE](#) $\triangleq$ image of $P^\bullet$
- ▶ Relationship with co-reducibility

Summary

- ▶ Regular expressions (unary/binary star, 1-free/under-star-1-free $(*/\perp)$)
- ▶ Milner's process interpretation P /semantics $\llbracket \cdot \rrbracket_P$
 - ▶ P -/ $\llbracket \cdot \rrbracket_P$ -expressible graphs ($\leadsto$ expressibility question)
- ▶ Layered Loop existence and elimination (LLEE/1-LLEE)
 - ▶ interpretation/extraction correspondences with $(*/\perp)$ /all reg. expr.
 - ▶ LLEE is preserved by bisimulation collapse (1-LLEE is not)
- ▶ TERMGRAPH 2024: compact process interpretation $P^\bullet$
 - ▶ image of $P^\bullet$ is closed under bisimulation collapse (image of P is not)
- ▶ $\text{LLEE} \triangleq$ image of $P^\bullet$ restricted to $(*/\perp)$ expressions
- ▶ $\text{1-LLEE} \triangleq$ image of $P^\bullet$
- ▶ Relationship with co-reducibility

IWC and IFIP-1.6 Talks

IWC (Friday, 24 July)

CG: *Loop Elimination in Process Graphs is Confluent When Pruning Steps are Added*

Ext. Abstract <https://clegra.github.io/lf/le-conf-IWC-26.pdf>

- ▶ critical-pair like analysis of loop elimination steps $\xrightarrow{\text{elim}}$
- ▶ $\xrightarrow{\text{elim}}$ forks are joinable by $\xrightarrow{\text{elim}}$ steps and prune steps $\xrightarrow{\text{prune}}$

IFIP-1.6 meeting (yesterday)

CG: *Finding Small Steps for Hard Problems, in Work on the Process Interpretation of Regular Expressions*

Slides https://clegra.github.io/lf/IFIP-1_6-2026.pdf

- ▶ Reducing for Graphs Representing Regular Expressions under Bisimilarity

Resources

- ▶ Slides/extended abstract on clegra.github.io
 - ▶ slides: [.../1f/TG-26.pdf](#)
 - ▶ extended abstract: [.../1f/pi-struc-TG-26.pdf](#)
- ▶ CG: Closing the Image of the Process Interpretation of 1-Free Regular Expressions Under Bisimulation Collapse
 - ▶ TERMGRAPH 2024, [extended abstract](#)
- ▶ CG: The Image of the Process Interpretation of Regular Expressions is Not Closed under Bisimulation Collapse
 - ▶ [arXiv:2303.08553](#), 2021/2023.
- ▶ CG: Milner's Proof System for Regular Expressions Modulo Bisimilarity is Complete
 - ▶ LICS 2022, [arXiv:2209.12188](#), [poster](#).
- ▶ CG, Wan Fokkink: A Complete Proof System for 1-Free Regular Expressions Modulo Bisimilarity,
 - ▶ LICS 2020, [arXiv:2004.12740](#), [video on youtube](#).
- ▶ CG: Modeling Terms by Graphs with Structure Constraints
 - ▶ TERMGRAPH 2018, [EPTCS 288](#), [arXiv:1902.02010](#).